



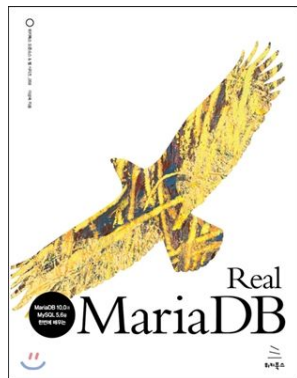
maria DB

목차

- 01장: MariaDB
- 02장: 설치
- 03장: MariaDB 기동 및 쿼리 실행
- 04장: 실행 계획 분석
- 05장: 최적화
- 06장: 스토리지 엔진
- 07장: 기타 기능
- 08장: 레플리케이션

2021.06.14

솔루션1팀 김은실



목차

■ 03장: MariaDB 기동 및 쿼리 실행

_3.1 서버 시작 및 종료

_3.2 서버 로그인

_3.3 데이터베이스 및 사용자 생성

_3.4 테이블 생성 및 변경

_3.5 데이터 조작

▣ 03장: MariaDB 기동 및 쿼리 실행

_3.1 서버 시작 및 종료

1. 서비스로 등록 → 시작 & 종료

- 자동 등록: 리눅스(RPM) / 윈도우(MSI) 설치 프로그램 이용.
- 운영체제의 각 부트업 레벨에 따라 서버가 적절히 종료되거나, 자동으로 재시작 되기를 원하는 경우.
- 일반적.

2. 서비스로 등록하지 않고 → 시작 & 종료

- 여러버전이 있고, 간단하게 테스트용으로: 데몬 모드를 원치않는 경우.
(개인적으로 설치: 자동으로 서버가 재시작되는 것을 원치 않는 경우,
굳이 서비스 등록이 귀찮은 경우, 자원낭비가 싫은 경우.)
- 가끔 직접 MariaDB 서버를 수동으로 시작하는 형태가 필요한 경우.

■ 03장: MariaDB 기동 및 쿼리 실행

_3.1 서버 시작 및 종료

1. 서비스로 등록

- 리눅스

- RPM으로 서버 설치

(/etc/init.d/mysql 초기화 스크립트 파일 자동 설치)

> service mysql start

> service mysql status [OK/Failed]

> service mysql stop

※피드백

CentOS 7로 오면서 → system으로 변경.

- init.d → systemd

- service 프로그램 start

→ systemctl start 프로그램

- chkconfig 프로그램

→ systemctl enable 프로그램

- 직접 컴파일 / tar.gz

<https://blog.naver.com/andwise/221029670946>

> cp ~ (mysql.server 스크립트 -> /etc/init.d/mysql로 복사.)

> chkconfig mysql (서비스로 등록)

- mysqladmin 유틸리티 사용하여 종료

> mysqladmin -u root -p shutdown

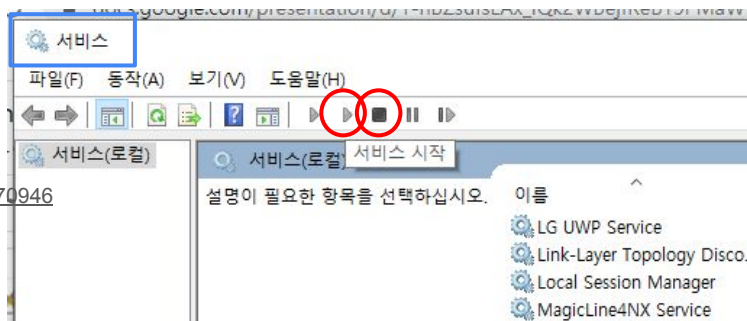
> mysqladmin -h 192.168.0.1 -P 3307 -u root -p shutdown

- 윈도우

(“서비스” 패널 이용)

- 설치 마법사 > 서비스 등록

> GUI



▣ 03장: MariaDB 기동 및 쿼리 실행

_3.1 서버 시작 및 종료

2. 서비스로 등록 X

- 리눅스

- 스크립트 만들

```
#!/bin/bash

MYSQL_HOME=/usr/local/mysql
${MYSQL_HOME}/bin/mysql_safe(엔젤프로세스)/mysql)
--defaults-file=/etc/my.cnf &
```

- support-files/mysql.server 스크립트

```
> sh ${MYSQL_HOME}/support-files/mysql.server start
> sh ${MYSQL_HOME}/support-files/mysql.server stop
```

- mysqladmin 유틸리티 사용 종료

```
> mysqladmin -u root -p shutdown
> mysqladmin -h 192.168.0.1 -P 3307 -u root -p shutdown
```

- 윈도우

- 스크립트 만들 (bat 파일)

```
> c:\mysql\MariaDB10.0.6\bin\mysqld
--defaults-file=c:\mysql\MariaDB10.0.6\my.ini
```

- mysqladmin 유틸리티 사용 종료

```
> c:\mysql\MariaDB10.0.6\bin\mysqladmin
-h127.0.0.1 -P3306 -u root -p shutdown
```

▣ 03장: MariaDB 기동 및 쿼리 실행

_3.2 서버 로그인 -mysql 커맨드 라인 툴
-HeidiSQL GUI 툴 (MariaDB 설치 프로그램(MSI)와 함께 설치됨.)

-서버 로그인 / 버전확인

> c:\mysql\MariaDB10.0.6\bin\mysql -h127.0.0.1(생략가능) -P3306(생략가능) -u root -p

```
[root@localhost.localdomain bin]# /home/local/mariadb-linux-20201113/bin/mysql -u root -p
Enter password:
Welcome to the MariaDB monitor. Commands end with ; or \g.
Your MariaDB connection id is 310
Server version: 10.3.22-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> |
```

※ ezEKP 패키지 현재 버전: 10.3.22-MariaDB를 사용하고 있음.

(MySQL 설치 프로그램에 포함된 mysql 클라이언트로 MariaDB에 접속)

> c:\mysql\mysql5.6.15\bin\mysql -h127.0.0.1(생략가능) -P3306(생략가능) -u root -p

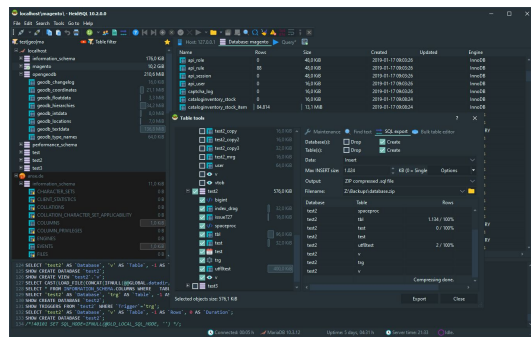
Server version: 5.5.5-10.0.6-MariaDB-log mariadb.org binary distribution

MariaDB는 가장 호환되는 버전인 5.5.5를 표시하고, 그 뒤 실제 MariaDB 서버의 버전인 10.0.6을 표시한다.

- 서버와 접속된 연결정보 확인: > \s > status

: 서버의 버전, 현재 클라이언트의 접속 계정, 문자 셋 변환 정보 출력.

※ HeidiSQL GUI란?



<https://blog.naver.com/harry5313/221441006993>

▣ 03장: MariaDB 기동 및 쿼리 실행

_3.2 서버 로그인

- 옵션

> mysql --help

-h(--host) -P(--port) -u(--user) -p(--password) -D(--database)

--auto-rehash : 테이블이나 컬럼명을 입력할 때 TAB으로 자동완성.

- 안전모드 : > ~ -U(--safe-updates)

UPDATE, DELETE, SELECT 모든 레코드를 업데이트 하거나, 모두 삭제하는 경우를 막아줌. (기준: 인덱스 사용하지 못할 때.)

- 비대화형 모드 : > ~ -e(--execute)="select * from A" ※ ezEKP 패키지 설치 시 유용하게 사용하고 있음.

-> 해당 쿼리를 실행하고 종료함. (cron / 스케줄러 작업용으로 유용.)

id	name	sex	address	birth
1	오징어	여자	청주	1988
2	꿀두기	남자	서울	1988
3	가오리	남자	경주	1988
4	코팍지	남자	제천	1977
6	이유태	남자	인천	1999
5	지리리	남자	연동	1988

- 배치모드 : --batch (실행모드와 같이 사용)

-> id name

1 A 형식으로 출력됨

(셀 스크립트를 이용해서 쿼리의 실행결과를 조작할 때 유용.)

- 컬럼제외 : -N(--skip-column-names)

(셀 스크립트에서 MariaDB 서버에 저장된 값을 읽어와서 뭔가를 처리할 때 유용.)

-> --batch + -e + -N

1 A

▣ 03장: MariaDB 기동 및 쿼리 실행

_3.3 데이터베이스 및 사용자 생성

- 사용자 식별

MySQL, MariaDB 만의 특징 : 사용자를 식별할 때, **host도 인증**의 일부가 됨.

같은 계정이라도, **host**가 다르다면 접속이 안될 수도 있고, 비밀번호가 다를 수도 있음.

%가 포함하더라도, 범위가 작은 (특정 ip)것 부터 인식한다.

```
MariaDB [mysql]> select user, host, password from user;
```

user	host	password
root	localhost	*3EC4983147BFEDCB8D2B3CE396F45432B2417E32
root	127.0.0.1	*3EC4983147BFEDCB8D2B3CE396F45432B2417E32
root	:::1	*3EC4983147BFEDCB8D2B3CE396F45432B2417E32
ezEKP2017	%	*3EC4983147BFEDCB8D2B3CE396F45432B2417E32
root	%	*3EC4983147BFEDCB8D2B3CE396F45432B2417E32

5 rows in set (0.005 sec)

▣ 03장: MariaDB 기동 및 쿼리 실행

_3.3 데이터베이스 및 사용자 생성

- 권한 GRANT

- (사용자까지 생성하고-IDENTIFIED BY pw) 권한 부여함. => mysql.user / mysql.host 에 저장

- ALL PRIVILEGES

grant sql: GRANT 권한(, ON 대상 오브젝트 (*/*DB).(*/*table/stored_program) TO 계정 ('user'@'host');

전역권한(글로벌 권한) *.*

CREATE USER (계정 생성) FILE (disk file 접근) (~WITH)GRANT OPTION (다른 사용자에게 권한 부여 가능)

PROCESS (MariaDB 서버내 프로세스 조회) RELOAD (FLUSH 명령 실행 가능)

REPLICATION CLIENT (복제 클라이언트 정보 조회) REPLICATION SLAVE (슬레이브 서버가 -> 마스터 서버에 접속)

SHOW DATABASES (DB목록 조회) SHUTDOWN (서버 종료) SUPER (제한을 넘어서서 작업 가능)

데이터베이스 권한 *.* DB.*

CREATE (DB 생성) CREATE ROUTINE (stored/function 생성) CREATE TEMPORARY TABLES (임시테이블 생성) DROP (DB 삭제)

EVENT (이벤트 생성/삭제) (~WITH)GRANT OPTION (DB 권한 -> 다른 사용자에게 권한 부여 가능) LOCK TABLES (명시적인 테이블 잠금)

테이블 레벨 권한 *.* DB.* DB.table

컬럼 레벨 권한

(GRANT SELECT, INSERT, UPDATE (a_column) ON ~ ; => update는 a_column 만 가능./ 권한체크 발생 => 성능↓ / 뷰 사용을 추천.)

ALTER (구조 변경) CREATE (테이블 생성(INDEX)) CREATE VIEW (뷰 생성) DELETE (데이터 삭제) DROP (테이블/뷰 삭제)

(~WITH)GRANT OPTION (table 권한 -> 다른 사용자에게 권한 부여 가능) INDEX (인덱스 생성) INSERT (데이터 저장)

SELECT (데이터 조회) SHOW VIEW (뷰의 구조 조회) TRIGGER (트리거 생성/삭제/실행) UPDATE (데이터 변경)

스토어드 프로그램(프로시저/함수) 레벨 권한 DB.stored

ALTER ROUTINE (스토어드 프로그램 내용 변경) EXECUTE (실행) (~WITH)GRANT OPTION (stored 권한 -> 다른 사용자에게 권한 부여 가능)

▣ 03장: MariaDB 기동 및 쿼리 실행

_3.3 데이터베이스 및 사용자 생성

- 권한 GRANT

- 권한 관리 지원(MariaDB 10.0.5 이상~: 권한그룹(Role)생성, 특정 사용자에게 권한 그룹 부여 가능.)

- GRANT sql - TO에 계정 대신 => 권한 그룹 사용 가능.

- 권한 그룹 생성

- mysql.user 테이블 : 권한 그룹 정보 저장. is_role = 'Y'로 설정 됨.

create role sql: CREATE ROLE 권한그룹이름(예약어 불가:PUBLIC, NONE)

(생략 가능) WITH ADMIN 권한 줄 수 있는 (CURRENT_USER(기본값), CURRENT_ROLE, user, role);

- 권한 그룹 부여

- mysql.roles_mapping 테이블 저장.

sql: GRANT 권한그룹이름 TO 계정;

- ~10.0.8버전 권한 그룹 자동 인식 X => SET

> select current_role; => null

sql: SET ROLE 권한그룹이름;

- 권한 그룹 2개 이상 불가: 마지막 set된 권한 그룹만 인식.

- 권한 그룹 제거

sql: SET ROLE NONE;

▣ 03장: MariaDB 기동 및 쿼리 실행

_3.3 데이터베이스 및 사용자 생성

- 기본 사용자

윈도우: MariaDB 홈\data -> 데이터 파일 저장.

리눅스: [mysql_install_db](#) 명령 -> 기본 데이터 생성.

```
'root'@'127.0.0.1'
```

```
'root'@'::1'
```

```
``@'localhost'
```

```
'root'@'localhost'
```

- localhost -> 접속 시 -h 옵션으로 명시해야 함.

- localhost, 127.0.0.1를 함께 만들어주는 것이 좋다.

- 새로운 MariaDB를 설치할 시 모든 계정을 삭제하고 다시 재설정하는 것을 권장.

- 보안적인 문제를 없애려면: [mysql_secure_installation](#) 유틸리티 -> 초기 db 생성.

- 기본 데이터베이스

- **mysql**: 사용자 인증 정보, 스토어드 프로그램, 이벤트 정보, 타임 존, 복제 등

- **information_schema**: 서버 시작 시 읽어들이는 메타 정보를 -> 테이블 형태로 접근할 수 있도록 해줌.

(이 db table들을 잘 이용하면, 스키마 정보, 스토어드 프로그램 정보, 인덱스 통계 정보 확인 가능)

- **performance_schema**: 쿼리를 처리할 때 발생하는 각종 이벤트, 잠금, 잠금 대기 현상들을 숫자로 기록하는 테이블들이 저장되어 있음.

디스크에는 테이블의 구조만 저장, 실제 데이터는 모두 메모리상에 존재, 서버 재시작시 모두 초기화.

- **test**: 테스트용 db (table empty)

> `select * from mysql.user\G` :test란 단어가 들어간 db는 누구든지 볼 수 있는 권한이 있다. 서비스용에서는 이 권한을 삭제함.

> `select * from mysql.db\G`

※피드백

information_schema: 메타 정보 관련

performance_schema: 성능에 관련

```
MariaDB [(performance_schema)]> use information_schema
Database changed
MariaDB [(information_schema)]> show tables;

+-----+
| Tables_in_information_schema |
+-----+
| ALL_PLUGINS                  |
| APPLICABLE_ROLES             |
| CHARACTER_SETS               |
| CHECK_CONSTRAINTS            |
| COLLATIONS                   |
| COLLATION_CHARACTER_SET_APPLICABILITY |
| COLUMNS                     |
| COLUMN_PRIVILEGES             |
| ENABLED_ROLES                 |
| ENGINES                       |
| EVENTS                       |
| FILES                         |
| GLOBAL_STATUS                 |
| GLOBAL_VARIABLES              |
| KEY_CACHES                    |
| KEY_COLUMN_USAGE              |
| PARAMETERS                    |
| PARTITIONS                    |
| PLUGINS                      |
| PROCESSLIST                   |
| PROFILING                     |
| REFERENTIAL_CONSTRAINTS       |
| ROUTINES                     |
| SCHEMATA                     |
| SCHEMA_PRIVILEGES             |
| SESSION_STATUS                |
| SESSION_VARIABLES             |
| STATISTICS                    |
| SYSTEM_VARIABLES              |
| TABLES                      |
+-----+
```

```
MariaDB [(none)]> use performance_schema
Database changed
MariaDB [(performance_schema)]> show tables;

+-----+
| Tables_in_performance_schema |
+-----+
| accounts                     |
| cond_instances                |
| events                       |
| events_stages_current         |
| events_stages_history         |
| events_stages_history_long   |
| events_stages_summary_by_account_by_event_name |
| events_stages_summary_by_host_by_event_name |
| events_stages_summary_by_thread_by_event_name |
| events_stages_summary_by_user_by_event_name |
| events_stages_summary_global_by_event_name |
| events_statements_current     |
| events_statements_history     |
| events_statements_history_long |
| events_statements_summary_by_account_by_event_name |
| events_statements_summary_by_digest |
| events_statements_summary_by_host_by_event_name |
| events_statements_summary_by_thread_by_event_name |
| events_statements_summary_by_user_by_event_name |
| events_statements_summary_global_by_event_name |
| events_waits_current          |
| events_waits_history          |
| events_waits_history_long     |
| events_waits_summary_by_account_by_event_name |
| events_waits_summary_by_host_by_event_name |
| events_waits_summary_by_instance |
| events_waits_summary_by_thread_by_event_name |
| events_waits_summary_by_user_by_event_name |
| events_waits_summary_global_by_event_name |
| file_instances                |
| file_summary_by_event_name    |
| file_summary_by_instance     |
| host_cache                    |
| hosts                         |
| mutex_instances               |
| objects_summary_global_by_type |
| performance_timers            |
| relock_instances              |
| session_account_connect_attrs |
| session_connect_attrs        |
| setup_actors                  |
+-----+
```

```
MariaDB [(information_schema)]> show PROCESSLIST;

+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Id | User | Host | db | Command | Time | State | Info | Progress |
+----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | system user | | NULL | Daemon | NULL | InnoDB purge worker | NULL | 0.000 |
| 2 | system user | | NULL | Daemon | NULL | InnoDB purge coordinator | NULL | 0.000 |
| 3 | system user | | NULL | Daemon | NULL | InnoDB purge worker | NULL | 0.000 |
| 4 | system user | | NULL | Daemon | NULL | InnoDB purge worker | NULL | 0.000 |
| 5 | system user | | NULL | Daemon | NULL | InnoDB shutdown handler | NULL | 0.000 |
| 147 | root | localhost | information_schema | Query | 0 | Init | show PROCESSLIST | 0.000 |
+----+-----+-----+-----+-----+-----+-----+-----+-----+
rows in set (0.000 sec)
```

※ 유용 **show**

PROCESSLIST;

- **time**: 초단위

어떤 **sql문**이 과도한지 확인.

▣ 03장: MariaDB 기동 및 쿼리 실행

_3.3 데이터베이스 및 사용자 생성

- DB 생성/삭제

> CREATE DATABASE db이름 (DEFAULT CHARACTER SET 문자 셋(utf8mb4));

- 문자 셋 생략할 경우: MariaDB 서버에 설정된 기본 문자셋, 기본 콜레이션이 적용됨.

> DROP DATABASE db이름;

※ 콜레이션 (collation): 정렬순서 sort

- 서비스 중인 서버는 주의: 용량이 많으면 오래걸릴 수 있음.

- USER 생성

- CREATE USER -> GRANT (없으면 USER 생성도 함)

- user / db 는 전혀 무관하게 관리되므로: db 생성 -> 이 db를 위한 user 생성, 권한부여가 일반적.

> GRANT 권한(.) ON 대상 오브젝트 TO 계정('user'@'host') IDENTIFIED BY 패스워드; (대상 오브젝트가 다른경우 쿼리를 따로 실행.)

> FLUSH PRIVILEGES; (:가끔 즉시 계정이 생성되지 않는 경우)

관리자 계정

- root는 잘 알려져 있으므로 별도의 관리자 아이디를 준비하는 것이 좋음.

- 보안상 예측하기 어려운 계정이 좋음.

- 모든 권한

- 로컬에서만 접속 가능하도록 localhost, 127.0.0.1만 허용해주는 것이 좋음.

■ 03장: MariaDB 기동 및 쿼리 실행

_3.4 테이블 생성 및 변경 DDL

※ American National Standards Institute (미국 국가표준 협회)

- 테이블 DDL -> 거의 표준화된 **ANSI SQL 문장** 사용

+ 옵션 (성능: (MariaDB 특성) 스토리지 엔진 명시, 로우(레코드) 포맷 지정 등)

- 테이블 생성

> CREATE TABLE (IF NOT EXISTS) 테이블명 (

.....

)(ENGINE=InnoDB);

- IF NOT EXISTS: (여부를 정확히 판단하기 어려운 경우) 테이블/컬럼이 이미 있는 경우 에러x -> 경고메세지.

- ENGINE: 스토리지 엔진 지정. (기본값: 설정파일(my.cnf / my.ini)에 명시된 기본 스토리지 엔진)

> SHOW CREATE TABLE 테이블명; (테이블 구조를 한눈에 보기 좋음. (복합 인덱스 등 파악))

> DESC 테이블명; (DESCRIBE, 깔끔하게 보임)

■ 03장: MariaDB 기동 및 쿼리 실행

_3.4 테이블 생성 및 변경 DDL

- 테이블 변경(오프라인) : ~MariaDB 5.5 온라인 스키마 변경 지원하지 않음.

- ALTER TABLE : 컬럼,인덱스 추가, 삭제

(CREATE INDEX, DROP INDEX로 인덱스 추가,삭제 가능하나 -> ALTER로 모두 모아서 처리 가능하기 때문에 더 유용함.)

- 항상 중요한 작업 뒤에는 출력되는 **중복**이나 **경고성 메시지**를 확인하는 것이 좋다.

- 과정 (→ 서비스 중단 사항)

테이블 잠금 → 임시테이블 생성 → 한 건씩 모두 복사함. → 테이블 삭제 / RENAME → 잠금 해제

(※ 테이블,컬럼:이름변경, 숫자타입:길이변경, ENUM타입:아이템추가, 코멘트 변경은 이 과정을 거치지 않음.)

=> (MariaDB) 여러개의 ALTER는 하나로 모아서 실행할 수 있도록 지원.

> ALTER TABLE test ADD a_column DATETIME NOT NULL.
ADD INDEX a_index (a_column);

▣ 03장: MariaDB 기동 및 쿼리 실행

_3.4 테이블 생성 및 변경 DDL

- 테이블 변경(온라인) : MariaDB 10.0, MySQL 5.6~ 온라인 스키마 변경 기능 도입.

> ALTER ONLINE TABLE ~

1. MariaDB

- MariaDB 5.3~.5 : - 레코드 복사가 필요한 작업(→ERROR) 인지 아닌지 확인하는 용도.
 - 레코드의 복사가 필요치 않은 경우에만 스키마 변경이 실행되도록 하는 안전 장치.
(테이블, 컬럼:이름변경, 숫자타입:길이변경, ENUM타입:아이템추가, 코멘트 변경)
- MariaDB 10.0~ : 테이블 구조 변경이 실행되는 중에도 → 다른 커넥션의 DML이 즉시 처리.
 - 10.0.8에서 가능한 DDL: 컬럼, 인덱스 추가, 삭제, 컬럼 순서변경.
 - (온라인 스키마 변경 지원) 스토리지 엔진: InnoDB, TokuDB

■ 03장: MariaDB 기동 및 쿼리 실행

_3.4 테이블 생성 및 변경 DDL

- 테이블 변경(온라인)

> ALTER TABLE ~, LOCK=옵션, ALGORITHM=옵션;

2. MySQL (=> MariaDB 호환.)

- LOCK 옵션: 테이블에 대한 동시성 처리에 대한 튜닝 기능 제공.

- DEFAULT: 사용안함. (NONE -> SHARE -> EXCLUSIVE 순으로 DDL 구문에 맞게 자동으로 선택)

- NONE: (잠금 걸지 않음) 읽기, 쓰기.

- SHARED: (읽기 잠금) 읽기.

- EXCLUSIVE: (배타적 잠금) 읽기, 쓰기. 한 개의 트랜잭션만 걸 수 있음.

- ALGORITHM 옵션: 온라인 DDL의 성능 관련된 튜닝 기능 제공.

- DEFAULT: 사용안함.

- INPLACE: 온라인과 비슷.

-과정: 지원되는 스토리지 엔진의 테이블인지 **확인** → 변경 **준비** (인덱스 정보 준비, 작업 중 변경되는 데이터 추적)

→ 변경 **작업** (작업 중 발생하는 DML → 별도의 로그 기록) → 로그 적용 → **COMMIT**

- 온라인 변경 로그: (디스크 아닌) 메모리에 생성. (※ innodb_online_alter_log_max_size 시스템 변수를: 잘 고려해서 실행해야 한다.

소요시간이 많을 경우, 공간 부족으로 취소될 수도 있음.)

- COPY: 오프라인 방식. 복사 → rename.

→ old_alter_table 시스템 변수 = ON으로 설정할 수도 있음. / ALGORITHM 옵션이 더 우선함.

■ 03장: MariaDB 기동 및 쿼리 실행

_3.4 테이블 생성 및 변경 DDL

- 테이블 변경(온라인)

> ALTER TABLE ~ , LOCK=옵션, ALGORITHM=옵션;

- ALGORITHM 옵션

- INPLACE

- 레코드 복사가 되느냐 안되느냐의 기준보다는

변경 작업 중 → 다른 커백션의 DML처리를 허용하는지 아닌지의 기준 정도로 생각하는 것이 좋다.

- INPLACE 알고리즘으로 처리, 다른 커백션 DML 실행, SELECT 실행

:인덱스 추가, 삭제 등 대부분 가능

- REORGANIZE: INPLACE 알고리즘으로 처리되지만, 레코드 복사.

:전문 인덱스 추가 (FULLTEXT INDEX 생성)

- REBUILD: INPLACE 알고리즘으로 처리 불가하면서 레코드 복사.

:(SELECT만 가능) 컬럼의 타입 변경, 프라이머리 키 삭제, FORCE 옵션+REBUILD 실행, 컬럼 문자셋 변경,정의

- 실패하는 경우.

- ALTER TABLE 장시간 실행 + 다른 커백션에서 DML 많이 실행되는 경우 / 온라인 변경 로그 공간 부족한 경우.

- DML이 변경 이후 테이블 구조에 적합하지 않다면, 마지막 과정에서 실패.

- 필요한 잠금 수준보다 낮은 옵션인 경우.

- 타임 아웃 발생 (lock_wait_timeout 시스템 변수 / ※대기 타임 아웃: innodb_lock_wait_timeout 과는 무관함.)

> SHOW GLOBAL VARIABLES LIKE 'lock_wait_timeout';

- (tmpdir 시스템 변수 → 디스크) 임시 디렉터리 공간이 부족한 경우.

▣ 03장: MariaDB 기동 및 쿼리 실행

_3.4 테이블 생성 및 변경 DDL

- 테이블 변경(온라인)

- **pt-online-schema-change** : 온라인 스키마 변경을 위한 3rd party 도구. (Percona에서 개발)

- 널리 사용.

- COPY와 비슷하지만, DML에 트리거 사용, 레코드를 청크 단위로 복사.(chunk-size 옵션)

- (주의사항)

- 트리거 중복 (DML에 사용할 ALTER트리거를 생성해야 하는데, 이미 ALTER 트리거가 있다면 → 실행 안됨.)

- 데드 락 (트리거를 통해서 들어오는 쿼리가 프라이머리 키가 아닌 경우 → (복)유니크 키에 배타적 잠금, (원)프라이머리 키에 잠금)

→ 데드 락의 경우 InnoDB/XtraDB: 두 트랜잭션 중 변경 수가 적은 쪽(pt-online..5~600건/사용자 1~2건)을 강제종료.

- 프라이머리 키나, 유니크 키가 없는 경우: 사용불가(DELETE 트리거에 필요.)

→ 테이블 파티션의 경우: 파티션키 제약사항 때문에 프라이머리키, 유니크 키를 가지고 있지 않은 경우가 많음.

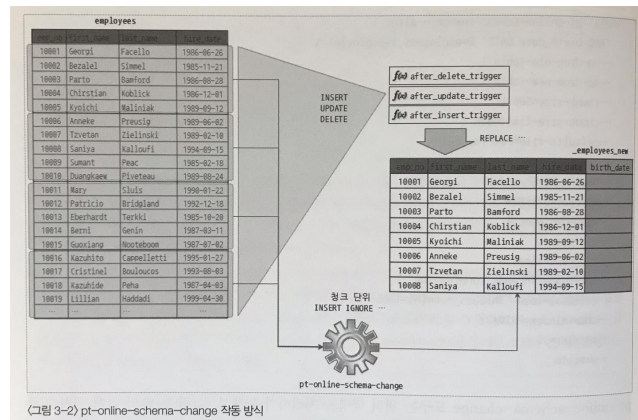
- 부하를 자동 조절할 수 있는 옵션: --max-load, --critical-load

- 조금 느려도 괜찮다면, 청크 복사가 완료될때마다 2~30ms 정도의 대기를 하는 코드를 넣어 변경해주면 좋음.

use Time::HiRes qw(time sleep usleep);

....

usleep(20000);



▣ 03장: MariaDB 기동 및 쿼리 실행

_3.4 테이블 생성 및 변경 DDL

- 테이블 변경(온라인)

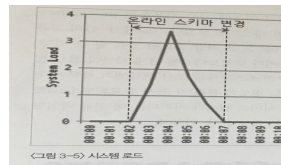
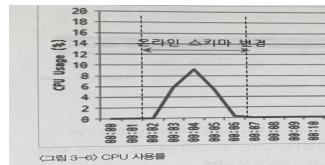
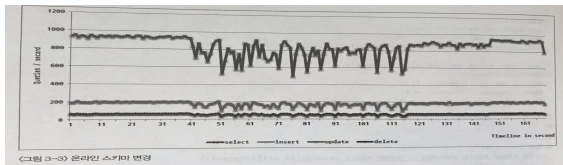
- 온라인 스키마 변경을 사용해도 될까?

- 저자가 MariaDB 10.0.6, MySQL 5.6.15 버전을 테스트한 결과: 성능은 아직 **pt-online-schema-change, OSC(facebook)** 도구가 더 안정적이라고 함.

- 온라인 스키마 변경: - 쿼리처리량이 불안하게 움직이고 있음.

- 시스템로드, CPU 사용률이 높음.

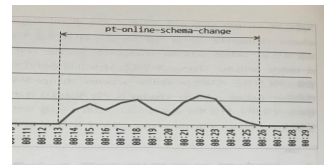
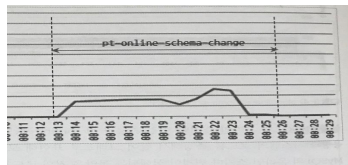
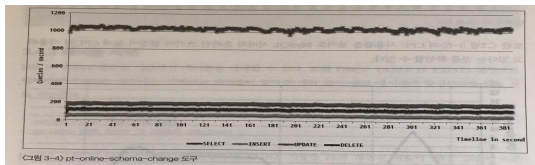
- 다른 커백션의 DML 쿼리를 허용하면서도, 빠르게 처리되는 대신에
자원을 많이 사용 → 서비스 쿼리에 악영향을 미칠 수 있음.



- (지원되기 이전 사용되었던) **pt-online-schema-change** 도구 (sleep-time - 코드 변경 상태):

- 서비스 쿼리에 악영향을 미치지 않음.

- 시스템로드, CPU 사용률도 낮은 수준에서 머무르고 있음.



▣ 03장: MariaDB 기동 및 쿼리 실행

_3.4 테이블 생성 및 변경 DDL

- 테이블 삭제

> DROP TABLE (IF EXISTS) 테이블명;

- (주의사항)

- **버퍼 풀**에서 해당 테이블의 페이지를 제거.

- 버퍼 풀에 캐싱된 테이블 데이터, 인덱스 페이지 등을 제거하기 위해서 → 버퍼 풀의 모든 캐시 페이지 스캔. (작업시간 많이 소모)
→ 버그 레포팅 등 개선된 이력들이 있어 특별히 문제가 되지 않을 수 있음.

- **디스크**에서 물리적인 데이터 파일 삭제.

- 특히 **EXT3** 파일 시스템에서 파일 삭제(**unlink** 시스템 콜 방식) (작업시간 많이 소모: 큰 파일의 경우 거의 마비 상태)
(**XFS** 파일 시스템은 상대적으로 빠름)
- 테이블 크기가 크다면, 삭제하는 작업도 새벽시간대를 이용하는 것이 좋음.

▣ 03장: MariaDB 기동 및 쿼리 실행

_3.5 데이터 조작 DML

- 반드시 대상 DB의 이름이 필요하지만, 기본 DB를 명시할 수 있다.

> USE DB명;

- INSERT : 레코드 저장

- 테이블 존재

> INSERT INTO 테이블명 (컬럼1(..)) VALUES (값1(..)) (ON DUPLICATE KEY UPDATE 컬럼a=값a);

- 테이블의 구조의 변경을 대비해, 컬럼을 나열하는 형태를 사용하는 것이 좋다.

- ON DUPLICATE KEY UPDATE 옵션: 중복된 레코드가 이미 존재할 때는 UPDATE가 실행, 아니면 INSERT.

(ex. 컬럼1(pk)의 경우: 컬럼a를 값a로 업데이트.)

- SELECT : 레코드 조회

> SELECT (* / 컬럼) FROM 테이블명 (WHERE 조건);

- mysql 클라이언트를 사용하는 경우: 행 단위로 컬럼 표시하기

> SELECT * / 컬럼 FROM 테이블명 \G (; 대신)

```
***** 1. row *****
      Host: %
       Db: insa
      User: ezEKP2017
Select_priv: Y
```

▣ 03장: MariaDB 기동 및 쿼리 실행

_3.5 데이터 조작 DML

- UPDATE : 레코드 변경

- 삭제하지 않고 일부를 변경함.
- WHERE 조건을 빼먹어 전체가 UPDATE되지 않도록 주의.
- > UPDATE 테이블명 SET 컬럼1=값1(,) (WHERE 조건);

- REPLACE

- 중복된 경우: delete and insert
- INSERT INTO ... ON DUPLICATE KEY UPDATE .. (insert or update) 와 비슷한 처리이긴 하지만 내부적으로는 완전히 다름.
- 일을 두번하기 때문에: 부하도 높고, 시스템 자원을 많이 소모.
- 가능하면 INSERT INTO ... ON DUPLICATE KEY UPDATE .. 를 사용하는 것이 좋다.

- DELETE : 레코드 삭제

- WHERE 조건을 빼먹어 전체가 UPDATE되지 않도록 주의.
- > DELETE FROM 테이블명 (WHERE 조건);

※ AUTO-COMMIT

- MariaDB, MySQL 기본적으로 AUTO-COMMIT 모드 작동.
- (실수를 대비하는 방법)
 - 백업 (오래되지 않도록)
 - AUTO-COMMIT 모드를 끄고, DELETE, UPDATE를 실행할 것을 권장함.



maria DB

목차

▣ 04장: 실행 계획 분석

감사합니다.